

CSED211: Data Lab1 Report

컴퓨터공학과 20240940 장해영

1. Overview

주어진 연산자만을 이용하여 정수에 대한 2가지의 연산, floating point에 대한 4가지 연산을 수행하는 과제이다.

2. Question #1. negate

A. Explanation

주어진 정수의 부호를 반대로 변경하는 것이다. 과제의 조건에서 -사용을 금지하였기 때문에 negation을 주어진 연산자로 표현하는 것이 중요하다.

B. Solution

과제의 조건에 주어진 연산자만을 이용하여 negation을 표현하기 위해서 two's complement에서 inverse를 이용한 식인 $\sim x + 1 = -x$ 를 이용하는 것이다. 이 식을 이용하면 $\sim$ 와 $+$ 연산자 만으로 negation을 표현할 수 있다.

C. Implementation

```
int negate(int x) {  
    return ~x + 1;  
}
```

새로운 변수를 만들 필요 없이, 바로 $(\sim x) \& (\sim y)$ 를 return하여 주어진 정수의 부호가 반전된 값을 반환하도록 하였다.

3. Question #2. isLess

A. Explanation

주어진 정수 x, y 에 대해 $x < y$ 인 경우 1을, $x \geq y$ 인 경우 0을 return하는 문제이다. if문과 $<$, $>$ 와 같은 연산자는 사용할 수 없기에 x 와 y 의 부호가 다른 경우, 같은 경우로 나누어 문제를 해결하는 편이 좋을 것 같다.

B. Solution

우선 x 와 y 의 부호가 다른 경우에는 overflow가 발생할 수 있기에 x 와 y 의 부호가 다른 경우, 같은 경우로 나눈다.

- x와 y의 부호가 다른 경우

x의 부호가 음수라면, y의 부호가 양수일 테니, $x < y$ 가 되어 1을 반환해야 한다. 반대로, x의 부호가 양수라면, y의 부호가 음수일 테니 $x > y$ 가 되어 0을 반환해야 한다.

이를 위해 x와 y의 부호비트만 남기기 위해 모두 오른쪽으로 31칸씩 shift한 후 1과 AND 연산을 수행하여 마지막 비트만을 남긴다. 이렇게 마지막 비트만을 남기는 이유는 반환할 때 0 또는 1을 반환해야 하므로 처음부터 하나의 비트로 연산을 진행하는 것이다.

이렇게 만들어진 부호비트를 이용하여 $\text{sign_x} == 1, \text{sign_y} == 0$ 인 경우 1을, $\text{sign_x} == 0, \text{sign_y} == 1$ 인 경우 0을 반환할 수 있도록 AND연산자와 NOT연산자로 두 부호비트를 연결해준다.

- x와 y의 부호가 같은 경우

이 경우, x-y를 진행해도 overflow가 발생하지 않으므로 x-y를 계산한 후 결과값의 부호를 이용하여 대소를 비교하면 된다. "x와 y의 부호가 같은 경우"와 "x-y의 값이 음수인 경우"를 모두 만족하는 경우 1을 반환하도록 두 조건을 AND연산자로 연결해준다.

C. Implementation

```
int isLess(int x, int y) {
    int sign_x = (x >> 31) & 1;
    int sign_y = (y >> 31) & 1;

    int diff_sign_result = (sign_x & !sign_y);

    int sign_x_negate_y = ((x + (~y + 1)) >> 31) & 1;
    int same_sign_result = !(sign_x ^ sign_y) & sign_x_negate_y;

    return (diff_sign_result | same_sign_result);
}
```

x와 y의 부호가 다른 경우와 같은 경우를 각각 계산하여 이를 OR연산자로 연결하였다.

4. Question #3. float_abs

A. Explanation

해당 문제는 single-precision 형식으로 입력된 값 uf를 적절히 조작하여 uf의 절댓값을 반환하는 것이 목적이다. 값이 존재하는 일반적인 경우, 절댓값을 반환하면 되지만, 수가 아닌 것(NaN)은 입력값 그대로 출력하여야 한다.

Single-precision 형식을 알기 위해서 numerical form에 대해 먼저 알아야 한다. Numerical form은, 소수를 $(-1)^s \times M \times 2^E$ 의 형태로 나타낸 것을 의미한다. 이 때 M

의 범위는 $[1.0, 2.0)$ 이다. 이 형식을 이용하여 s, m, e 의 값을 unsigned int로 저장하여 소수를 표현하는 방법을 IEEE 754라 칭한다. 구체적인 형식은 다음 그림과 같다.



S는 부호비트인 s , e 는 E, f 는 M의 역할을 수행한다. Single-precision은 32bits를 이용하는데, s 는 1bit, e 는 8bits, f 는 23bits를 사용한다.

B. Solution

주어진 수의 절댓값으로 나타내는 것은 어렵지 않다. 최상위 비트가 부호비트 이므로 이 비트만으로 바꿔주는 연산을 하면 된다. 그렇지만, 해당과제에서는 NaN인 경우와 그렇지 않은 경우를 구분지어 다른 결과값을 반환해야 한다.

Single-precision 형식에서 NaN인 경우는 $e == 11111111 \ \&\& \ f \neq 0$ 이므로 e 와 f 값을 추출해야한다.

e 값은 right shift를 이용하여 정보가 담긴 값(e 값)을 오른쪽으로 이동시킨 뒤 해당 위치의 비트만을 추출하기 위해 최하위 비트부터 1이 8개 있는 값인 0xFF와 AND연산을 진행하여 추출할 수 있다.

f 값은 이미 가장 오른쪽에 있기 때문에 최하위 비트부터 1이 23개 있는 값인 0x007FFFFF와 AND연산을 하여 추출할 수 있다.

이렇게 추출된 값을 이용하여 NaN인 경우, 원래값을 그대로 반환하고, NaN이 아닌 경우(수가 존재하는 경우)는 부호비트를 0으로 바꾼 값을 반환하면 된다.

C. Implementation

```
unsigned float_abs(unsigned uf) {
    unsigned int exp = (uf >> 23) & 0xFF;
    unsigned int frac = uf & 0x007FFFFF;

    if (exp == 0xFF && frac != 0){
        // if uf is NaN return uf
        return uf;
    } else {
        return uf & 0x7FFFFFFF;
    }
}
```

e 값과 f 값을 판단하기 위해 각각 $exp, frac$ 변수를 이용하여 값을 저장하여 연산을 진행하였다. NaN의 조건인 $e == 11111111 \ \&\& \ f \neq 0$ 을 만족하는 경우, 입력값을 그대로 반환하고, 조건을 만족하지 않는 경우, 부호비트를 0으로 바꾼 값을 반환하기 위해 0x7FFFFFFF와 AND연산을 진행하였다.

5. Question #4. float_twice

A. Explanation

해당 문제는 소수로 표현할 수 있는 수라면 2를 곱한 값을, 소수로 표현할 수 없는 값이라면 입력값을 그대로 반환하는 문제이다. 여기서 소수로 표현할 수 없는 값이라 하면, ∞ 와 같이 무한대이거나, $0 \times \infty$ 와 같이 Not a Number 즉, 수가 아닌 값들을 말한다. 위 3번 문제와 같이 입력값을 그대로 반환하는 수를 구분해내는 코드가 필요할 것이다.

B. Solution

3번에서 사용한 방법과 마찬가지로 exp부분과 frac부분을 나누어준다. 다만, 3번에서는 무조건 양수로 반환하면 되었기에 부호비트는 신경쓰지 않아도 되었지만, 이번 문제에서는 부호는 그대로 유지해야 하기 때문에 부호비트도 추출해준다. Single-precision에서 부호비트는 최상위비트에 위치해 있기 때문에 주어진 uf와 0x80000000에 대해 AND연산을 진행해주면 최상위비트에 부호정보가 저장되고, 나머지 비트는 모두 0으로 표현된 값을 얻을 수 있다. 추후 이 값을 이용하여 연산을 진행한다.

이번 문제에서는 return값을 3가지 종류로 나눌 수 있다. ①NaN ②denormalized ③normalized 이다.

① NaN

해당 경우는 아무런 연산을 진행하지 않고 단순히 입력값을 그대로 반환하면 되므로 uf를 return한다.

② Denormalized

denormalized의 경우 조건인 $e == 0$ 를 만족시키는 경우이므로 fraction부분에 2를 곱해주기 위해 left shift를 한 칸 해주면 된다. exp부분은 어차피 0이기 때문에 uf를 그대로 시프트해주고, 부호비트만 다시 제대로 붙여주면 되므로 시프트를 진행한 후 부호비트와 AND연산을 진행한다.

③ Normalized

normalized의 경우 exp부분에 1을 더해주면 된다. exp부분이 2의 n제곱인 경우 n를 나타내기 때문에 1을 더해준 후 다시 원래의 자리로 시프트해주기 위해 left shift를 23칸 해준 뒤 미리 구해놓은 부호비트, fraction부분과 연결해주면 된다.

C. Implementation

```
unsigned float_twice(unsigned uf) {
    unsigned int sign = uf & 0x80000000u;
    unsigned int exp = (uf >> 23) & 0xFF;
    unsigned int frac = uf & 0x07FFFFFF;

    // if uf is NaN or infinity
    if (exp == 0xFF) {
        return uf;
    }
    // if uf is denormalized
    else if (exp == 0) {
        return (uf << 1) | sign;
    }
    // if uf is normalized
    else {
        exp += 1;
        return sign | (exp << 23) | frac;
    }
}
```

입력받은 uf를 이용하여 미리 sign, exp, frac부분을 구해놓은 뒤 경우에 따라 연산을 진행하도록 하였다.

부호비트를 추출할 때 0x80000000u 에서 u를 사용한 이유는 unsigned임을 명확히 하기 위해서이다.

6. Question #5. float_i2f

A. Explanation

정수형태로 입력받은 x를 floating point의 형태로 나타내는 과제이다. 이 때 IEEE방식의 single precision의 형태로 나타내면 된다. 주어진 정수를 float의 형태로 나타내는 것이기에 NaN나 무한대는 고려하지 않아도 된다.

B. Solution

우선 single precision의 형태로 나타내기 위해 주어진 정수의 sign, exp, frac을 파악해야 된다.

- Sign

해당 과제에서는 부등호를 사용해도 되기 때문에 $x < 0$ 로 판단할 수 있다. IEEE방식에서 음수이면 1, 양수이면 0으로 표현하므로 $x < 0$ 인 경우 참이라는 조건을 이용한다.

- Exp

해당 부분은 주어진 정수의 절댓값을 비트로 나타내었을 때 1의 최상위비트의 자릿수를 계산하여 나타낼 수 있다. 따라서 1이 나올 때까지 해당 값을 오른쪽으로 시프트해주고, 시프트한 수를 세면 자릿수를 계산할 수 있다. 이렇게 계산한 자릿수와 bias를 더하면 exp에 해당하는 수이다. Bias는 $2^{e-1} - 1$ 로 나타낼 수 있

는데, 이 때 e는 exponent bits, 즉 이 문제에서는 8 이므로 bias는 127이다. 따라서 127과 위에서 계산한 자릿수를 더해주면 된다.

- Frac

exp에 해당하는 수까지 모두 구한 후 남은 수를 소수의 형태로 나타내는 부분이다. 실제로 남은 수가 1.xxxxxxxx..가 될 것인데 이 exp에는 소수점위에 있는 1은 나타내지 않아도 되므로 이 1을 제거해준 변수를 아래 코드에서 abs_remove_hidden_1이라 표현하였다. Single precision으로 나타낼 것이기에 이 값은 23자리까지 표현될 수 있다. 따라서 23자리를 초과한다면, rounding을 통해 값을 반올림해주는 작업이 필요하다. 이 때 3가지의 경우가 존재한다.

- ① 24번째 비트가 0인 경우: 이 경우에는 0.5보다 작다는 의미이므로 버린다.
 - ② 24번째 비트가 1이고, 그 다음 비트들 중에 1이 있는 경우: 0.5보다 크다는 의미로 올린다.
 - ③ 24번째 비트가 1이고, 그 다음 비트들 중에 1이 없는 경우: 0.5이므로 round to even규칙을 적용한다. 이 round to even은 마지막 비트가 짝수 즉 0이 되도록 반올림을 진행하는 것이다. 즉 마지막 비트가 1이면 올리고, 0이면 내린다.
- 이와 같이 반복하다 계속 올림이 발생하여 최상위비트 위에 비트가 추가되면 frac을 0으로 만들고 exp의 값을 1만큼 증가시킨다.

위의 과정을 모두 마친 뒤 sign, exp, frac의 값을 제자리로 정렬한 후 OR연산자로 연결해 준다.

C. Implementation

```
unsigned float_i2f(int x) {
    unsigned int sign = 0, exp, frac;
    unsigned int abs = x, abs_remove_hidden_1, copy_x;
    unsigned int msb_pos=0, extra, half;

    // if x is 0
    if (!x){
        return x;
    }

    // find abs
    if (x < 0){
        abs = ~x+1;
        sign = 1;
    }

    // find msb
    copy_x = abs;
    while (copy_x >>= 1){
        msb_pos += 1;
    }

    // compute exp, frac
    exp = 127 + msb_pos;
    abs_remove_hidden_1 = abs ^ (1u << msb_pos);

    if (msb_pos < 23){
        frac = abs_remove_hidden_1 << (23 - msb_pos);
    } else {
        // rounding
        int shift = msb_pos - 23;
        frac = abs_remove_hidden_1 >> shift;
        extra = abs_remove_hidden_1 & ((1u << shift) - 1);
        half = 1u << (shift - 1);

        if ((extra > half) || (extra == half && (frac & 1))){
            if(++frac >> 23){
                frac = 0;
                exp += 1;
            }
        }

        // sign + exp + frac
        return (sign << 31) | (exp << 23) | frac;
    }
}
```

입력값인 정수 x가 0인 경우 floating point로 나타낸 경우도 0이므로 그대로 반환하는 로직을 추가하였다.

7. Question #6. float_f2i

A. Explanation

5번 문제와 반대로 floating-point로 주어진 값을 int형태로 반환하는 문제이다. IEEE 형태로 주어졌기 때문에 NaN이나 무한대를 고려하여 코드를 구성해야한다.

B. Solution

3번에서의 방법과 마찬가지로 exp, frac 부분을 추출한다. 다만, 이번 문제의 exp의 경우 bias를 고려하여 127을 추가로 빼준다. Bias는 $2^{(e-1)}-1$ 로 나타낼 수 있는데, 이 때 e는 exponent bits, 즉 이 문제에서는 8 이므로 bias가 127이다. 또한, sign비트를 추출하기 위해 오른쪽으로 31만큼 시프트해준다. 이러면 모든 비트가 sign비트로 채워지게 되어 부호를 판단할 수 있다.

첫번째 경우인 NaN과 무한대인 경우 0x80000000u를 반환하기 위해 이 둘의 조건인 $e == 0xFF$ 인 경우, 이 코드에서 exp를 제대로 계산하기 위해 빼주었으므로 127을 더 해준다. 즉, $(exp + 127) == 0xFF$ 인 경우 0x80000000u를 반환해준다.

두번째 경우로, 지수파트가 0미만이라 0의 값을 반환해야하는 경우이다. 즉 $exp < 0$ 이면 0을 반환하면 된다.

세번째 경우로, exp가 31이상인 경우 overflow가 발생하는 경우이다. Exp가 31이라면, 실제 int로 나타내야 하는 수가 2^{31} 이상이 되어 overflow가 발생한다. 이 경우 또한 0x80000000u를 반환하도록 하였다.

마지막으로 일반적인 경우이다. Exp가 23초과인 경우 frac이 더 커져야 하므로 frac을 $exp - 23$ 만큼 왼쪽으로 시프트해준다. 반대로, exp가 23이하인 경우 frac이 더 작아져야 하므로 frac을 $23 - exp$ 만큼 오른쪽으로 시프트해준다. 마지막으로 이렇게 시프트된 값에 위에서 구한 부호를 붙여 반환한다.

C. Implementation

```
int float_f2i(unsigned uf) {
    unsigned int sign = uf >> 31;
    int exp = ((uf >> 23) & 0xFF) - 127;
    unsigned int frac = (uf & 0x007FFFFF) | 0x800000;

    // if uf is NaN or infinity
    if ((exp + 127) == 0xFF) {
        return 0x80000000u;
    }

    // if exp < 0 -> 0
    else if (exp < 0) {
        return 0;
    }

    // if exp >= 31 -> overflow
    else if (exp >= 31) {
        return 0x80000000u;
    }

    // if normalized
    else if (exp > 23) {
        frac <<= (exp - 23);
    } else {
        frac >>= (23 - exp);
    }

    if (sign) {
        return -frac;
    } else {
        return frac;
    }
}
```

4가지의 경우로 나누어 각 경우에 맞는 값을 반환하도록 하였다.